

C++ Classes: Access Control and Header Files

Based on ECM2433 L12 Classes, cppIntro, and L13 Memory slides. Covers private vs protected access, getters and setters, the this pointer, splitting classes across .h/.hpp and .cpp files, include guards, and separate compilation with g++.

What You Need To Know

1. *private, protected, and public*

The key distinction from L12: private members are accessible only within the class itself. protected members are accessible within the class **and** within any derived class, but not from outside code. public members are accessible by anyone. A class defaults to private; a struct defaults to public.

```
class Base {
    int i;          /* private: only Base's own functions */
protected:
    int j;          /* protected: Base + derived classes */
public:
    void set(int a, int b) { i = a; j = b; }
    void show() { printf("i=%d, j=%d\n", i, j); }
};

class Derived : public Base {
    int k;
public:
    Derived(int x) { k = x; j = 0; } /* j OK: protected */
    void showAll() { show(); }      /* show() OK: public */
    /* i = 5; would be ERROR: private to Base */
};

int main() {
    Derived d(3);
    d.set(1, 2); /* OK: public */
    d.show();    /* OK: public */
    /* d.j = 5; ERROR: protected, not accessible from main */
}
```

2. *Getters and setters*

When a member is private, external code cannot access it directly. Provide a public **getter** to read it and a public **setter** to write it. The setter can validate input before applying it.

```
class Temperature {
    double celsius;
public:
    double get_celsius() const { return celsius; }
    void set_celsius(double c) {
        if (c < -273.15) return; /* reject impossible value */
        celsius = c;
    }
};
```

3. *The this pointer*

this is a pointer to the current object. It is used to disambiguate when a constructor parameter has the same name as a member variable.

```
class NewClass {
    int a, b;
public:
    NewClass(int a, int b) {
        this->a = a; /* this->a: member; a: parameter */
        this->b = b;
    }
};
```

4. *Splitting a class across .h and .cpp*

The header file declares the class (what it has). The .cpp file defines the member functions (what they do), using the scope-resolution operator :: to name the class they belong to.

```
/* --- counter.h --- */
#ifndef COUNTER_H
#define COUNTER_H

class Counter {
    int count;
public:
    Counter();
    void increment();
};
```

```

        int get_count() const;
    };

#endif

/* --- counter.cpp --- */
#include "counter.h"

Counter::Counter() { count = 0; }

void Counter::increment() { count++; }

int Counter::get_count() const { return count; }

```

5. Include guards

If a header is included twice in the same translation unit (e.g., via two different headers both including the same file), the contents are processed twice and structures are defined twice — a compile error. An include guard prevents this.

```

#ifndef STACK_H
#define STACK_H

/* content of stack.h */

#endif

```

6. Separate compilation and linking with g++

Only the essentials are repeated here: compile each .cpp file to a .o object file with g++, then link the object files into the final executable. The -std=c++17 flag selects the language standard.

```

$ g++ -std=c++17 -c counter.cpp -o counter.o
$ g++ -std=c++17 -c main.cpp -o main.o
$ g++ main.o counter.o -o myprogram

```

C++ headers may use the .h extension or .hpp (some libraries such as Boost prefer .hpp).

Practice Questions

- 1 Write the counter.h class declaration for a Counter with private int count and public Counter(), void increment(), and int get_count() const.
- 2 In counter.cpp, why must the function definition be written as void Counter::increment() rather than just void increment()?

Question 3 — fix the naming conflict:

```

class Box {
    int width;
public:
    Box(int width) {
        width = width; /* bug: assigns parameter to itself */
    }
};

```

- 3 Explain the bug and rewrite the constructor body using this to fix it.
- 4 Write a getter get_age() const and a setter set_age(int) for a private int age member. The setter must reject negative values.
- 5 Write the complete include guard for a header file called sensor.h.

Question 6 — this program does not compile:

```

/* grandparent.h */
struct Data { int x; };

/* parent.h */
#include "grandparent.h"

/* child.cpp */
#include "parent.h"
#include "grandparent.h" /* included again */
int main() { return 0; }

```

- 6 Explain the compile error and show what to add to grandparent.h to fix it.
- 7 Given a class Counter declared in counter.h with a private int count and public Counter(), void increment(), and int get_count() const, write the counter.cpp file that implements all three using the :: operator.

- 8 State the three g++ commands needed to compile counter.cpp and main.cpp (both including counter.h) into an executable called myprogram, using the C++17 standard.

Question 9 — the destructor is missing:

```
class Buffer {  
    char *data;  
public:  
    Buffer(int size) { data = new char[size]; }  
    /* destructor missing */  
};
```

- 9 Write the correct destructor for Buffer. Explain what happens if it is omitted.
- 10 What is the key difference between C's malloc and C++'s new when allocating an object of a class? What about free vs delete?

Mark Scheme

Attempt all questions before checking.

- 1 Expected header shape: `#ifndef COUNTER_H / #define COUNTER_H / class Counter { int count; public: Counter(); void increment(); int get_count() const; }; / #endif`.
- 2 `Counter::increment` says the function is the member function declared inside `Counter`. Without the scope-resolution prefix, the compiler sees a separate free function named `increment`.
- 3 The bug: `width = width` assigns the parameter to itself; the member is never set. Fix: `this->width = width;`
- 4 `int get_age() const { return age; }` and `void set_age(int a) { if (a >= 0) age = a; }`
- 5 `#ifndef SENSOR_H / #define SENSOR_H / ... header content ... / #endif`
- 6 The error is a redefinition of `struct Data` because `grandparent.h` is processed twice. Fix: add an include guard to `grandparent.h`: `#ifndef GRANDPARENT_H / #define GRANDPARENT_H / content / #endif`
- 7 `counter.cpp`: `#include "counter.h"`. Then: `Counter::Counter() { count = 0; } / void Counter::increment() { count++; } / int Counter::get_count() const { return count; }`
- 8 `g++ -std=c++17 -c counter.cpp -o counter.o` then `g++ -std=c++17 -c main.cpp -o main.o` then `g++ main.o counter.o -o myprogram`. Use `g++` rather than `gcc` so the C++ standard library is linked correctly.
- 9 `~Buffer() { delete[] data; }`. Without it, the heap memory allocated in the constructor is never freed (memory leak) every time a `Buffer` goes out of scope.
- 10 `malloc` allocates raw memory without calling any constructor. `new` allocates memory then calls the constructor. `free` releases memory without calling the destructor. `delete` calls the destructor then deallocates memory.